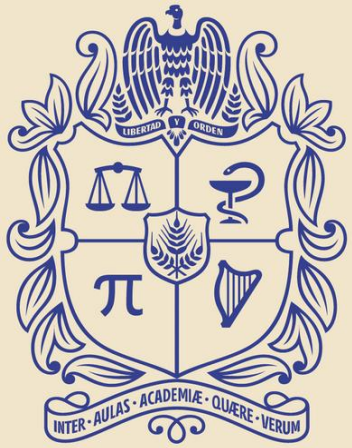


UNIVERSIDAD
NACIONAL
DE COLOMBIA

CURVE FITTING

Daniel Felipe Rodríguez Ramírez, MIG
dafrodriguezram@unal.edu.co

CIUDAD UNIVERSITARIA, BOGOTÁ D.C., MAY 2026

UNIVERSIDAD NACIONAL DE COLOMBIA

Facultad de INGENIERÍA Sede Bogotá

Genki

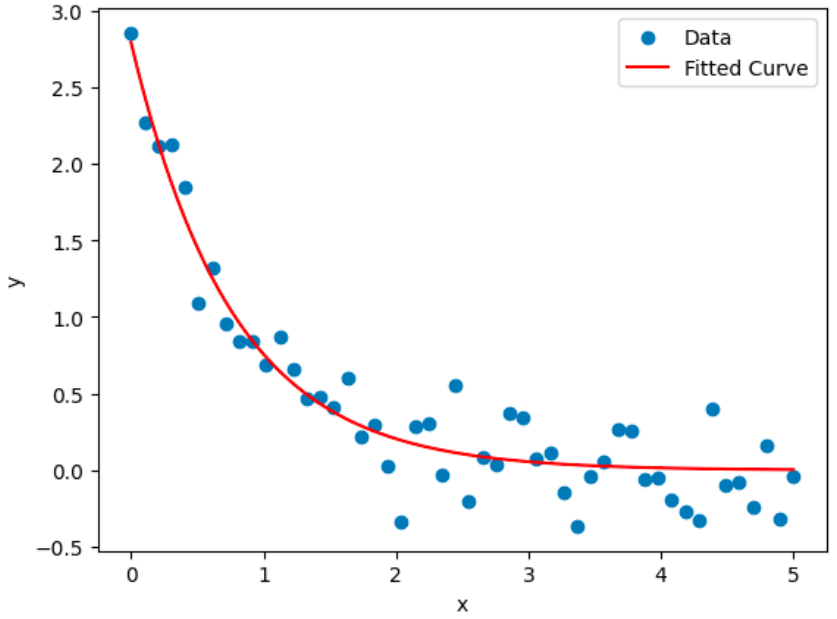
CONTENT

1. Introduction

Motivation

- Need for estimating intermediate values from discrete data.

- Two general approaches:
  - **Trend-fitting** (noisy data → least-squares regression)
  - **Interpolation** (precise data → exact passage through points)



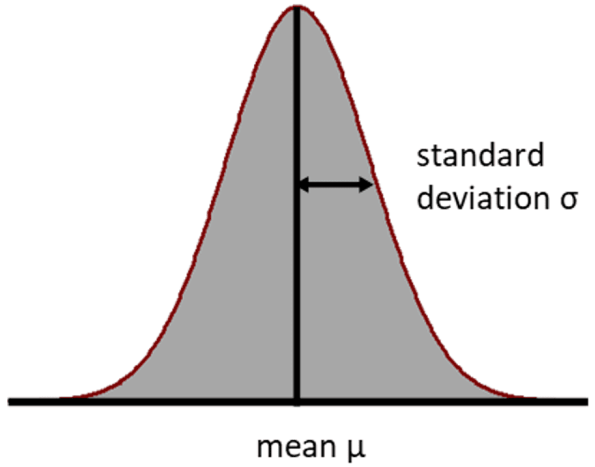
Mathematical Background

Simple Statistics

Arithmetic mean, variance, standard deviation, coefficient of variation

The Normal Distribution

- Histogram approximation by bell-shaped curve
- Relation between σ, μ, and probability intervals



Estimation of Confidence Intervals

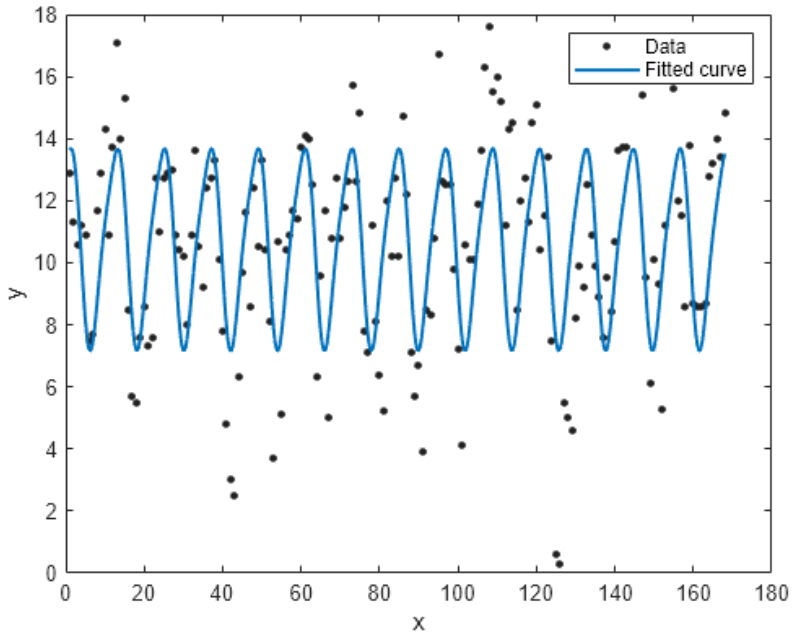
z- and t-distributions for two-sided intervals

Methods

Trade-offs
Comparative advantages and limitations of regression vs. interpolation

Important Methods
Key methods for regression, interpolation, Fourier fits, confidence intervals

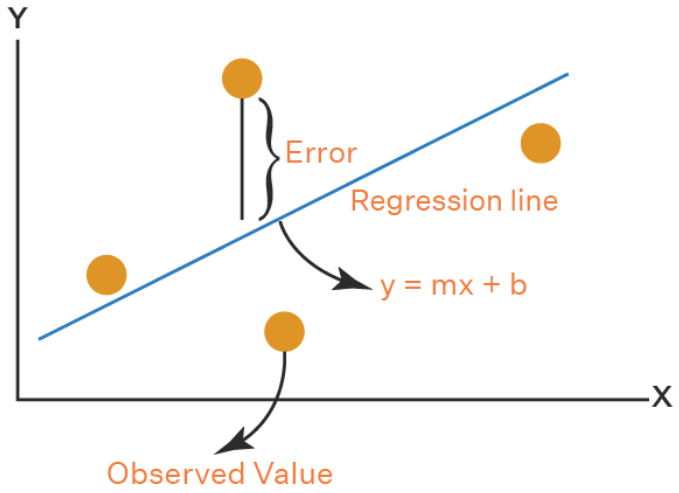
Advanced Methods
Matrix formulations, general linear models, software-based implementations



2. Least-Squares Regression

Motivation for Least-Squares Regression

- Experimental data often noisy; high-order interpolating polynomials oscillate and yield poor predictions between points.
- A simpler trend-fitting function (a straight line) can capture the overall behavior without overfitting.

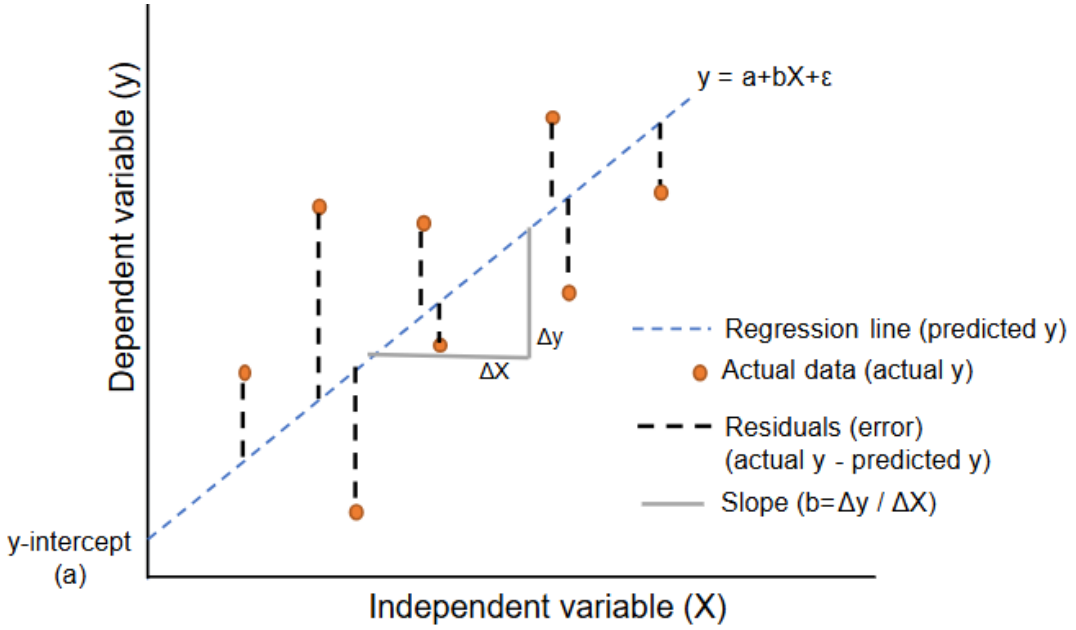


Linear Model and Residuals

- Model: $y = a_0 + a_1 x + e$

- Residual (error) for each point (x_i, y_i):

$$e_i = y_i - (a_0 + a_1 x_i)$$



Closed-Form Solutions

Slope

$$a_1 = \frac{n\sum x_i y_i - \sum x_i \sum y_i}{n\sum x_i^2 - (\sum x_i)^2}$$

Intercept

$$a_0 = \bar{y} - a_1 \bar{x}$$

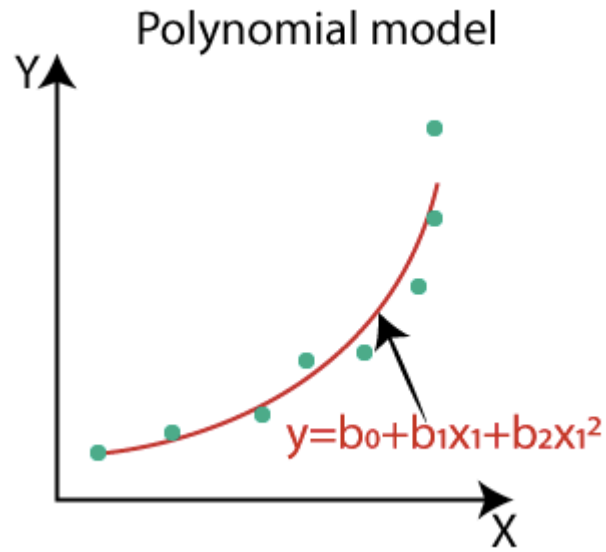
Extension to Higher-Order Polynomials

Instead of fitting

$$y = a_0 + a_1 x + e$$

we fit an mth-order polynomial

$$y = a_0 + a_1 x + a_2 x^2 + \cdots + a_m x^m + e$$



Extension to Higher-Order Polynomials

$$(n)a_0 + \left(\sum x_i\right)a_1 + \left(\sum x_i^2\right)a_2 = \sum y_i$$

$$\left(\sum x_i\right)a_0 + \left(\sum x_i^2\right)a_1 + \left(\sum x_i^3\right)a_2 = \sum x_i y_i$$

$$\left(\sum x_i^2\right)a_0 + \left(\sum x_i^3\right)a_1 + \left(\sum x_i^4\right)a_2 = \sum x_i^2 y_i$$

Extension to Higher-Order Polynomials

Numerical Considerations

- The normal-equation matrix can be ill-conditioned, especially for high m.
- Round-off errors may lead to inaccurate coefficients.
- Remedies include:

– Employing higher-precision arithmetic
– Limiting to lower polynomial orders when possible.

Example 1

Fit a second-order polynomial to the following data

x_i	y_i
0	2.1
1	7.7
2	13.6
3	27.2
4	40.9
5	61.1
Σ	152.6

3. Interpolation

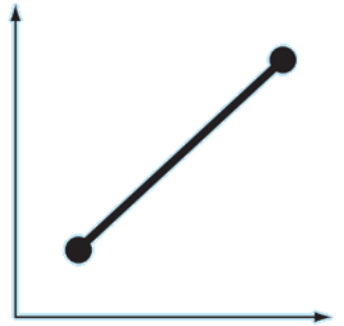
Overview

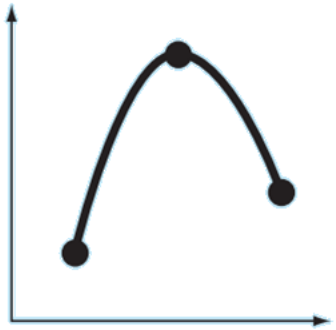
- You often need to estimate intermediate values between precise data points.

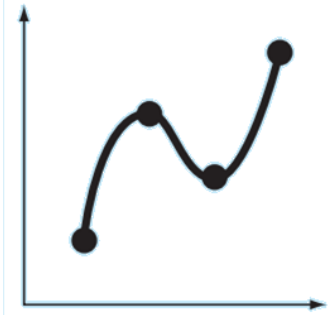
- The most common method is polynomial interpolation, where an nth-order polynomial

$$f(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$$

passes exactly through n+1 given data points.







Linear Interpolation (First-Order Case)

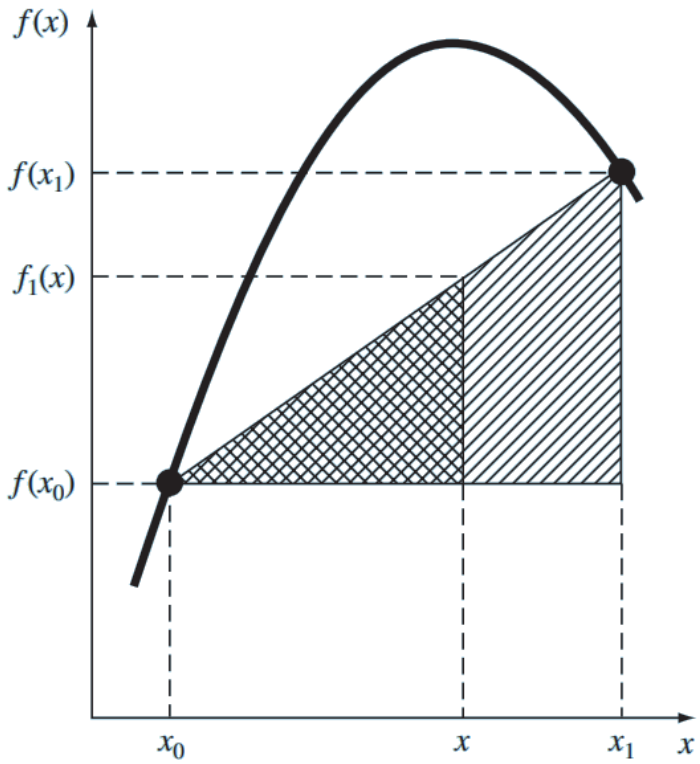
- The simplest interpolation connects two data points $(x_0, f(x_0))$ and $(x_1, f(x_1))$ with a straight line.

- From similar triangles:

$$\frac{f_1(x) - f(x_0)}{x - x_0} = \frac{f(x_1) - f(x_0)}{x_1 - x_0}$$

- Rearranged (linear-interpolation formula):

$$f_1(x) = f(x_0) + \frac{f(x_1) - f(x_0)}{x_1 - x_0}(x - x_0)$$



Example 2

Estimate the natural logarithm of 2 using linear interpolation. First, perform the computation by interpolating between ln 1 = 0 and ln 6 = 1.791759. Then, repeat the procedure, but use a smaller interval from ln 1 to ln 4 (1.386294). Note that the true value of ln 2 is 0.6931472.

Quadratic Interpolation (Three Points)

Newton's form:

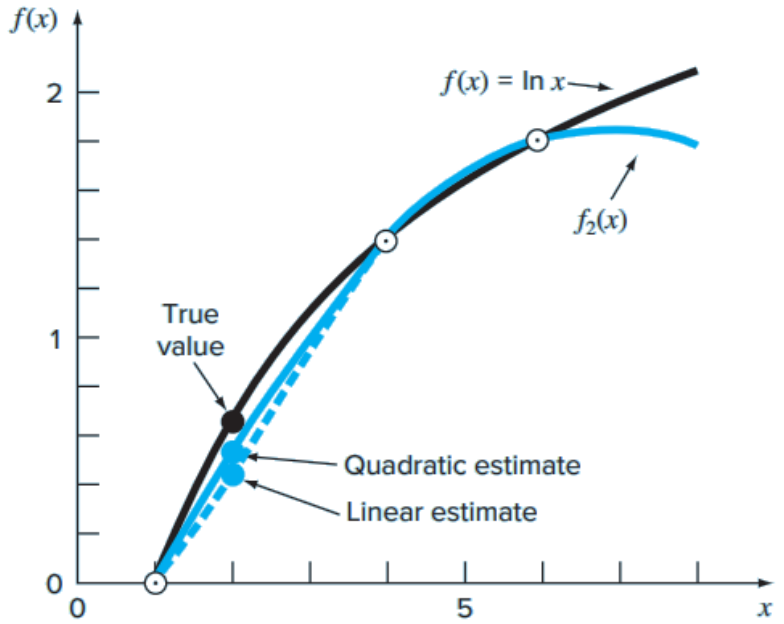
$$f_2(x) = b_0 + b_1(x - x_0) + b_2(x - x_o)(x - x_1)$$

Coefficient formulas:

$$b_0 = f(x_0)$$

$$b_1 = \frac{f(x_1) - f(x_0)}{x_1 - x_0}$$

$$b_2 = \frac{\frac{f(x_2) - f(x_1)}{x_2 - x_1} - \frac{f(x_1) - f(x_0)}{x_1 - x_0}}{x_2 - x_0}$$



Example 3

Fit a second-order polynomial to the following three points:

$$x_0 = 1 \qquad f(x_0) = 0$$
$$x_1 = 4 \qquad f(x_1) = 1.386294$$
$$x_2 = 6 \qquad f(x_2) = 1.791759$$

General Newton's Divided-Difference Interpolating Polynomial

Objective: Fit an *n*th-order polynomial through *n*+1 points

$[x_0, f(x_0)], \ldots, [x_n, f(x_n)]$

Polynomial form:

$$f_n(x) = b_0 + b_1(x - x_0) + b_2(x - x_0)(x - x_1) + \cdots + b_n(x - x_0) \ldots (x - x_{n-1})$$

Divided-difference coefficients:

$$b_0 = f(x_0)$$
$$b_1 = f[x_1, x_0]$$
$$b_2 = f[x_2, x_1, x_0]$$
$$\vdots$$
$$b_n = f[x_n, x_{n-1}, \ldots, x_1, x_0]$$

$$f[x_i, x_j] = \frac{f(x_i) - f(x_j)}{x_i - x_j}$$

$$f[x_i, x_j, x_k] = \frac{f[x_i, x_j] - f[x_j, x_k]}{x_i - x_k}$$

$$f[x_n, x_{n-1}, \ldots, x_1, x_0] = \frac{f[x_n, x_{n-1}, \ldots, x_1] - f[x_{n-1}, x_{n-2}, \ldots, x_0]}{x_n - x_0}$$

General Newton's Divided-Difference Interpolating Polynomial

i	x_i	$f(x_i)$	First	Second	Third
0	x_0	$f(x_0)$	$f[x_1, x_0]$	$f[x_2, x_1, x_0]$	$f[x_3, x_2, x_1, x_0]$
1	x_1	$f(x_1)$	$f[x_2, x_1]$	$f[x_3, x_2, x_1]$	
2	x_2	$f(x_2)$	$f[x_3, x_2]$		
3	x_3	$f(x_3)$			

Example 4

In Example 3, data points at $x_0 = 1, x_1 = 4$, and $x_2 = 6$ were used to estimate ln 2 with a parabola. Now, adding a fourth point, $[x_3 = 5; f(x_3) = 1.609438]$, estimate ln 2 with a third-order Newton's interpolating polynomial

Lagrange Interpolating Polynomial

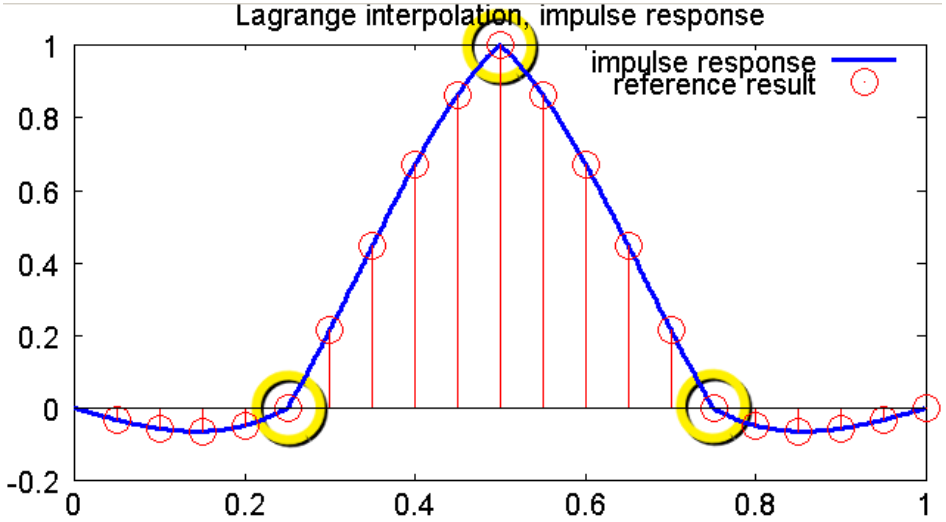
- **Definition:** A reformulation of Newton's polynomial that avoids computing divided differences.

- **General form:**

$$f_n(x) = \sum_{i=0}^{n} L_i(x) f(x_i)$$

- **Basis polynomials $L_i(x)$:**

$$L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^{n} \frac{x - x_j}{x_i - x_j}$$



Example 5

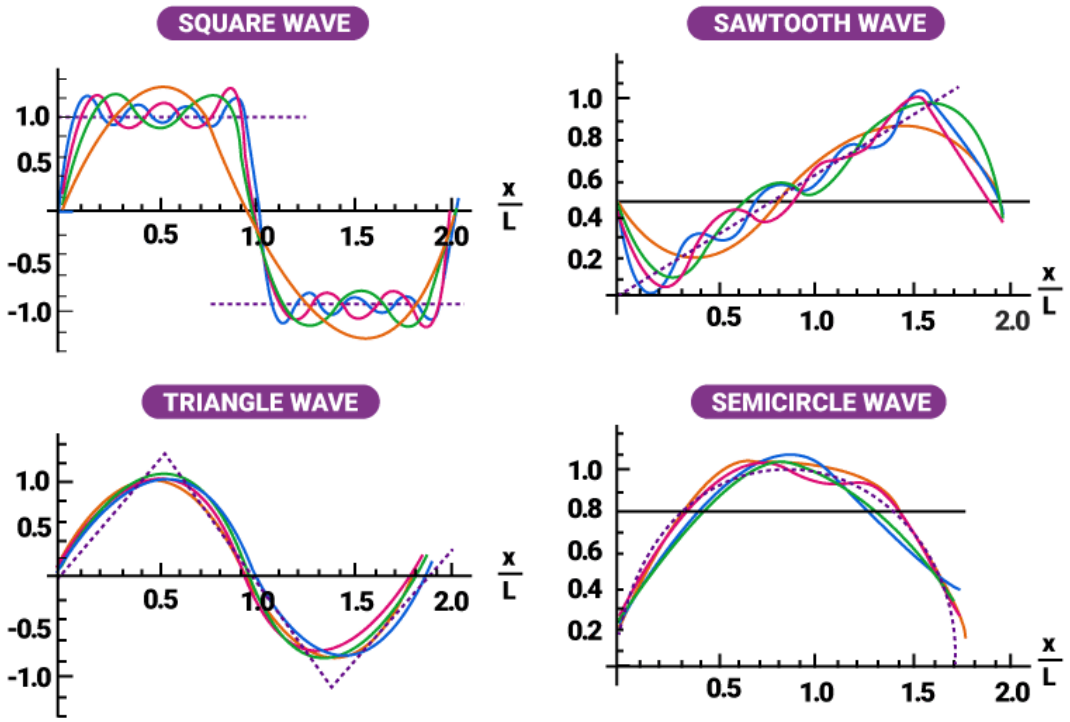
Use a Lagrange interpolating polynomial of the first and second order to evaluate ln 2 based on the following data:

$$x_0 = 1 \qquad f(x_0) = 0$$
$$x_1 = 4 \qquad f(x_1) = 1.386294$$
$$x_2 = 6 \qquad f(x_2) = 1.791760$$

4. Fourier Approximation

Fourier Approximation

A systematic framework for representing arbitrary waveforms using trigonometric (sine/cosine) series.

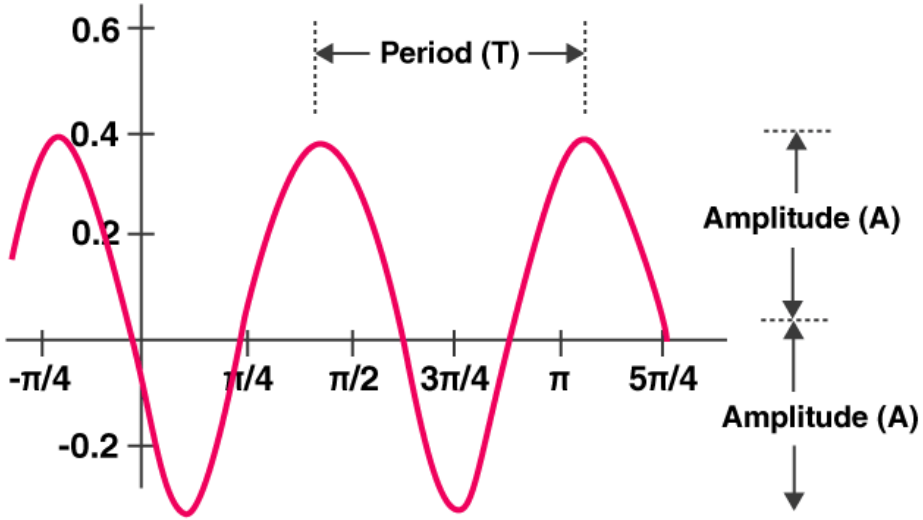


Periodic Functions

Definition: *f(t)* is periodic if

$$f(t) = f(t + T)$$

Where T is the (smallest) period.

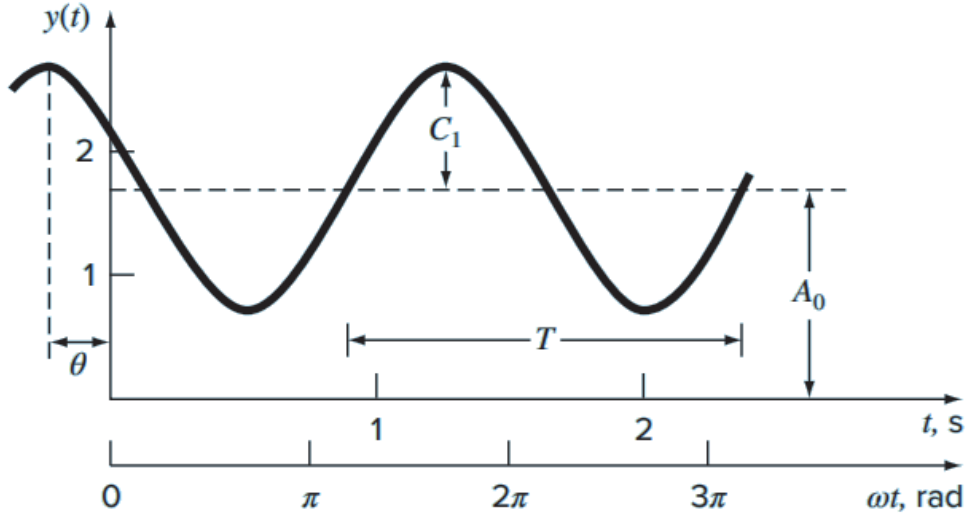


Sinusoidal Model

General form:

$$f(t) = A_0 + C_1 \cos(\omega_0 t + \theta)$$

Parameters:
A_0: mean (DC) value
C_1: amplitude
ω_0: angular frequency



Least-Squares Fit of a Single Sinusoid

Model:

$$y_i = A_0 + A_1 \cos(\omega_0 t_i) + B_0 \sin(\omega_0 t_i) + e_i$$

Normal Equations (Matrix Form)

$$\begin{bmatrix} N & \Sigma\cos(\omega_0 t) & \Sigma\sin(\omega_0 t) \\ \Sigma\cos(\omega_0 t) & \Sigma\cos^2(\omega_0 t) & \Sigma\cos(\omega_0 t)\sin(\omega_0 t) \\ \Sigma\sin(\omega_0 t) & \Sigma\cos(\omega_0 t)\sin(\omega_0 t) & \Sigma\sin^2(\omega_0 t) \end{bmatrix} \begin{Bmatrix} A_0 \\ A_1 \\ B_0 \end{Bmatrix} = \begin{Bmatrix} \Sigma y \\ \Sigma y \cos(\omega_0 t) \\ \Sigma y \sin(\omega_0 t) \end{Bmatrix}$$

Example 6

The sinusoid curve is described by $y = 1.7 + \cos(4.189t + 1.0472)$. Generate 10 discrete values for this curve at intervals of Δt = 0.15 for the range t = 0 to 1.35. Use this information to evaluate the coefficients of by a least squares fit.

5. Curve Fitting with Software Packages

Excel Curve Fitting Tools

Trendline Tool:

- Supports linear, polynomial, logarithmic, exponential, power, and moving average fits.

Built-in Functions for Regression:

Function	Description
FORECAST	Returns a value along a linear trend
GROWTH	Returns values along an exponential trend
INTERCEPT	Returns the intercept of the linear regression line
LINEST	Returns the parameters of a linear trend
LOGEST	Returns the parameters of an exponential trend
SLOPE	Returns the slope of the linear regression line
TREND	Returns values along a linear trend

Excel Curve Fitting Tools

- **Data Analysis ToolPak:**
  Implements general linear least squares models of the form:
  $$y = a_0 z_0 + a_1 z_1 + a_2 z_2 + \cdots + a_m z_m + e$$
- **Solver Tool:**
  Used for nonlinear regression by minimizing squared residuals.

Example 7

You are asked to fit the given data series (pairs of x and y values) to the logarithmic model

$$y = a_0 + a_1 \, logx$$

using Excel's Trendline tool, in order to determine the coefficients a_0 and a_1 that best match the provided points.

x	y
0.5	0.53
1.0	0.69
1.5	1.50
2.0	1.50
2.5	2.00
3.0	2.06
3.5	2.28
4.0	2.23
4.5	2.73
5.0	2.42
5.5	2.79

MATLAB built-in functions for interpolation and transforms

Higher-dimensional interpolation functions

- interp2 and interp3 perform two- and three-dimensional piecewise interpolation, respectively, analogous in usage to interp1.

Function	Description
`polyfit`	Fit polynomial to data
`interp1`	1-D interpolation (1-D table lookup)
`interp2`	2-D interpolation (2-D table lookup)
`spline`	Cubic spline data interpolation
`fft`	Fast Fourier transform

Example 8

Generate a set of data points by sampling the function:

$$f(x) = \sin(x)$$

at equally spaced xx–values from 0 to 10 with a step size of 1. Then, using MATLAB, fit the resulting data $\{(x,y)\}$ by means of:

1. **Linear interpolation**
2. **A fifth-order polynomial**
3. **A cubic spline**

Finally, compare each fitted curve to the original sine data by evaluating and plotting them on a finer x-grid (for example, with step size 0.25) over the interval [0,10].

6. Summary

Summary

Motivation:

- Need to estimate intermediate values from discrete (often noisy) data.
- Two main approaches:
  - Trend-fitting (noisy data → least-squares regression)
  - Interpolation (precise data → exact passage through given points)

Key Methods Overview:

- Regression: fit a trend line or low-order polynomial to noisy data
- Interpolation: construct a function that passes exactly through data points
- Fourier fits: represent periodic data via sine/cosine series

Summary

Least-Squares Regression:

- Fit a model (e.g. straight line, polynomial) by minimizing the sum of squared residuals
- Linear and higher-order polynomial models

Interpolation Techniques:

- Linear interpolation (first-order) between consecutive data points
- Quadratic interpolation using three points
- Newton's divided-difference formulation for general-order polynomials
- Lagrange interpolating polynomials

Summary

Fourier Approximation:

- Systematic framework for modeling arbitrary periodic waveforms
- Representation via trigonometric series (sine and cosine terms)
- Least-squares fit of a single sinusoid to data

Curve Fitting Software Tools:

- **Excel:** Trendline tool supporting linear, polynomial, logarithmic, exponential, power, and moving-average fits; built-in regression functions
- **MATLAB:**
  - polyfit/polyval for regression and interpolation
  - fft for Fourier transforms and approximations
  - interp1, interp2, interp3 for one-, two-, and three-dimensional interpolation

Summary

Fourier Approximation:

- Systematic framework for modeling arbitrary periodic waveforms
- Representation via trigonometric series (sine and cosine terms)
- Least-squares fit of a single sinusoid to data

Curve Fitting Software Tools:

- **Excel:** Trendline tool supporting linear, polynomial, logarithmic, exponential, power, and moving-average fits; built-in regression functions
- **MATLAB:**
  - polyfit/polyval for regression and interpolation
  - fft for Fourier transforms and approximations
  - interp1, interp2, interp3 for one-, two-, and three-dimensional interpolation